

Track A: PushT를 위한 Scratch Visual Encoder

알고리즘, 수학적 목적함수, 실험 설계 및 평가 계획

AdaJEPA 실험 브랜치: `robust-encoder-scratch`

구현 기준일: 2026년 9월 27일

초록

본 문서는 AdaJEPA 저장소에 구현한 Track A 실험을 설명한다. 무작위 초기화한 vision transformer에 masked reconstruction, 동일 상태의 색상 변환 복원, corruption-to-clean 복원을 적용한다. 공유 decoder에는 목표 영상의 색상을 별도로 전달하고, encoder에는 외형 변화에도 유지되는 장면 정보를 학습하도록 요구한다. 정확한 손실함수, offline four-view 데이터셋, 교대 최적화, 고정 선형 probe, 비교군, 기록 및 재현 절차를 기술한다. 현재까지 확인된 것은 구현과 로컬 smoke test이다. 대규모 학습 결과, 실제 benchmark 향상, MPC 성공률 향상은 아직 입증하지 않았다. 이미 구현한 기능과 앞으로 수행할 실험을 구분해서 설명한다.

저장소: <https://github.com/jinu7513/adajepa>

검토한 기준 커밋: 66c5f18d00cc627ae35b9b183367f7e510972106

구현 기준 파일: `conf/tracka.yaml`, `tracka/`

상태: 구현 및 smoke 검증 완료, 과학적 실험 결과는 미확정.

목차

1 연구 질문과 실험 목적	3
1.1 왜 visual encoder를 연구하는가?	3
1.2 각 설계를 선택한 이유	3
1.3 범위와 주장할 수 있는 내용	3
2 기호와 관측 모델	4
3 Offline four-view 데이터셋	4
3.1 채택한 상태마다 정확히 네 개의 view	4
3.2 Canonical state와 reset 검증	5
3.3 연속 색상 domain randomization	5
3.4 연속 corruption 강도	6
3.5 Split, 상태 선택, metadata	6

4 Encoder와 공유 reconstruction decoder	7
4.1 Visible token만 처리하는 양방향 encoder	7
4.2 Mask token 삽입과 위치 복원	7
4.3 연속 목표 RGB conditioning	7
4.4 Tensor shape와 optional CLS	8
5 정확한 학습 목적함수	8
5.1 Masked reconstruction의 정의	8
5.2 Clean MAE baseline	8
5.3 Render 목적함수: clean 중심 양방향 복원	9
5.4 Corruption 목적함수: 손상 입력에서 clean 복원	9
5.5 Optional global reconstruction과 collapse 진단	9
6 최적화 알고리즘과 예산	10
6.1 네 가지 학습 모드	10
6.2 Alternating 알고리즘	10
6.3 공정한 비교의 의미	10
7 Frozen representation 평가	11
7.1 Physical-state ridge probe	11
7.2 Metric과 해석	11
7.3 Test 조건	12
7.4 Nuisance 진단과 reference	12
8 실험 계획법과 판단 기준	13
8.1 검증할 가설	13
8.2 주 실험 matrix	13
8.3 실행 순서와 ablation	13
8.4 어떤 결과가 가설을 지지하는가?	14
8.5 통계 보고	14
9 W&B 기록, 재현성, run 관리	15
10 구현 검증과 한계	15
A 실행 명령 순서	16
B 코드와 방법의 대응	17

1 연구 질문과 실험 목적

1.1 왜 visual encoder를 연구하는가?

PushT에서 계획을 세우려면 agent 위치, block 위치와 방향, 목표 도형의 정보를 영상에서 읽어야 한다. 물체 색상이 바뀌거나 적당한 blur가 추가되어도 실제 장면의 배치는 그대로일 수 있다. 그런데 representation이 크게 달라지면 predictor나 latent 거리 기반의 계획 목적함수가 작업에 필요한 기하 정보보다 외형에 반응할 가능성이 있다. Track A는 planning이나 dynamics predictor를 바꾸기 전에 이 문제를 representation 수준에서 분리해 조사한다.

핵심 질문은 다음과 같다. *PushT* 데이터만으로 처음부터 학습한 *encoder*가 선형적으로 읽을 수 있는 장면 기하 정보를 유지하면서, 색상 변화와 적당한 *corruption*에 덜 민감해질 수 있는가? 강건성 향상은 검증할 가설이지 구조만으로 보장되는 결과가 아니다.

1.2 각 설계를 선택한 이유

1. **Scratch 초기화**: 작업 데이터에서 어떤 표현을 학습하는지 조사한다. Frozen DINOv2는 외부 사전학습 reference이며, 같은 사전학습 데이터로 맞춘 대조군은 아니다.
2. **동일 상태의 정적 재렌더링**: 장면 상태를 거의 고정한 채 외형만 바꾼다. 따라서 같은 위치의 patch 사이에 의미 있는 대응이 생긴다.
3. **Masked reconstruction**: 보이지 않는 장면 부분을 복원할 정보를 representation에 요구한다. Visible token만 처리하는 encoder와 가벼운 decoder는 MAE의 설계 방식 [1]을 따른다. 본 실험의 50% mask와 별도 손실은 작업에 맞춘 선택이며, 원 논문의 전체 학습 recipe를 그대로 재현하는 것은 아니다.
4. **목표 색상 conditioning**: decoder가 필요한 색상을 직접 받으므로 복원을 위해 encoder가 입력 색상을 반드시 보존해야 하는 부담을 줄인다.
5. **Corruption 목적함수 분리**: 예측할 수 없는 noise realization 자체를 복원하도록 요구하지 않는다.
6. **Clean으로 학습한 고정 probe**: visual shift 이후에도 같은 readout이 작동하는지 평가한다. 조건마다 probe를 다시 학습해서 변화에 보상하는 효과를 막는다.

1.3 범위와 주장할 수 있는 내용

이 브랜치는 encoder 사전학습과 representation 평가를 구현한다. Flow-JEPA, flow matching, 새로운 MPC, test-time adaptation, policy는 구현 범위가 아니다. 이전에 수행한 last-block/last-two-block 또는 multistep AdaJEPA adaptation은 별도의 학습 및 평가 메커니즘이다. 여기서 *frozen*은 probe를 학습하는 동안 encoder를 고정하고, *shifted test*에서는 probe까지 고정한다는 뜻이다. 기존 논문의 Frozen 대 AdaJEPA planning 비교와 동일한 구분이 아니다.

기존 visual token의 shape를 맞추는 것은 인터페이스 호환 조건이다. 새 scratch latent space를 DINO 기반 predictor가 곧바로 이해한다는 뜻은 아니다. End-to-end planning 성능을 주장하려면 이후 별도의 연결 및 predictor 학습 절차가 필요하다.

2 기호와 관측 모델

현재 사용할 수 있는 simulator state는

$$s = (a_x, a_y, b_x, b_y, \theta, v_{a,x}, v_{a,y}) \in \mathbb{R}^7 \quad (1)$$

이다. Agent와 block의 위치, block 각도, agent 속도를 포함한다. Block의 선속도와 각속도 또는 완전한 물리 엔진 snapshot은 포함하지 않는다. Rendering nuisance는

$$\eta = (c_b, c_a, c_g), \quad c_o \in [0, 1]^3 \quad (2)$$

로 정의한다. 각 RGB는 block, agent, goal에 대응한다. Renderer를 R , corruption을 C 라 하면

$$x = R(s, \eta), \quad x_{\text{corr}} = C(R(s, \eta), \epsilon) \quad (3)$$

이다. ϵ 에는 난수 realization이 포함된다. Corruption 종류와 강도는 재현용 metadata로 저장하지만 encoder나 decoder에는 전달하지 않는다.

$E_\theta(x) \approx f(s)$ 라는 목표는 영상에서 관측할 수 있는 장면 정보를 안정적으로 표현하겠다는 뜻이다. 전체 simulator state를 역으로 복원할 수 있다는 뜻은 아니다. 위치는 같고 속도만 다른 두 상태는 같은 정지 영상을 만들 수 있다. 따라서 속도 probe는 보조 진단이며 visual sufficiency의 결정적인 기준이 아니다.

기호	의미
E_θ, D_ϕ	Scratch image encoder와 공유 reconstruction decoder
x_0, x_A, x_B, x_c	Clean, 두 색상 변형, clean의 corrupted image
$P = 196, Q = 768$	Patch 수와 16×16 RGB patch의 pixel scalar 수
$d = 384, d_D = 192$	Encoder 및 decoder embedding 차원
M_i, V_i	표본 i 의 masked/visible patch index 집합
Z_i^u	View u 의 visible patch representation
B, T	Batch 크기와 전체 optimizer update 예산
$\lambda_r, \lambda_c, \lambda_g$	Render/corruption/optional global loss 가중치

표 1: 핵심 기호. 수치는 저장소 기본 config 기준이다.

3 Offline four-view 데이터셋

3.1 채택한 상태마다 정확히 네 개의 view

선택한 source state에서 다음 영상을 생성한다.

$$x_0 = R(\bar{s}, \eta_0), \quad x_A = R(\bar{s}, \eta_A), \quad (4)$$

$$x_B = R(\bar{s}, \eta_B), \quad x_c = C(x_0, \epsilon). \quad (5)$$

$\bar{s}$ 는 검증한 reset 이후 canonical state이다. 서로 reset한 view 사이의 상태 일치는 설정한 수치 허용오차 안에서 성립한다. Corrupted image는 저장할 clean render에 직접 corruption을 적용해 만든다. Random-color image를 출발점으로 사용하지 않는다. 이미지는 uint8 PNG로 저장하고 loader에서 정규화한다.

Source pool은 `dataset.source_path`, `dataset.source_splits`로 명시한다. 후자의 기본값은 [train] 이다. 생성기는 state tensor와 유효 trajectory 길이를 읽고, shape metadata가 있으면 T 물체만 선택한 뒤 전부 재렌더링한다. Planning target인 `val_T/plan_targets.pkl`은 encoder 사전학습 데이터로 사용하지 않는다.

3.2 Canonical state와 reset 검증

현재 reset은 입력 state를 설정한 뒤 0.01 초의 physics step을 한 번 실행한다. 원본 상태를 s^{src} 라 할 때

$$\bar{s} = \text{Reset}(s^{\text{src}}, \eta_0), \quad s^q = \text{Reset}(s^{\text{src}}, \eta_q) \quad (6)$$

로 정의한다. 색상 view도 반드시 s^{src} 에서 reset한다. $\bar{s}$ 에서 다시 reset하면 추가 물리 진행이 생길 수 있다. 반환된 각 state를 원본과 canonical 두 기준에 모두 비교한다. 위치/속도 그룹에는 좌표별 최대 절대오차를 사용하고, 각도는

$$\delta_\theta = \text{atan2}(\sin(\theta - \theta_{\text{ref}}), \cos(\theta - \theta_{\text{ref}})) \quad (7)$$

로 비교한다. 하나라도 tolerance를 넘으면 해당 four-view sample 전체를 제외한다. Probe의 정답은 실제 렌더링 영상과 대응하는 $\bar{s}$ 에서 만든다.

항목	원본 대비 tolerance	Canonical 대비 tolerance
Agent 위치	5.0	10^{-4}
Block 위치	5.0	10^{-4}
Block 각도(rad)	0.1	10^{-4}
Agent 속도	10^{-3}	10^{-4}

표 2: 현재 후보 기본값. 위치와 속도는 simulator 단위이며 실제 데이터에서 제외율을 먼저 확인한다.

이 검증은 빠르게 움직이거나 접촉 중인 상태를 더 많이 제외할 수 있다. Generation report에는 필드별 최대 오차와 split/segment별 제외 수를 남긴다. 모든 sample을 통과시키기 위해 근거 없이 tolerance를 넓히기보다, 제외가 만드는 selection bias를 먼저 보고해야 한다.

3.3 연속 색상 domain randomization

각 물체의 HSV를 독립적으로 제안한 뒤 공동 rejection rule을 적용한다.

$$h \sim U(0, 1), \quad s_{\text{HSV}} \sim U(0.45, 0.90), \quad v_{\text{HSV}} \sim U(0.40, 0.85). \quad (8)$$

RGB로 변환한 값을 uint8로 반올림한다. 거리 검사는 HSV 공간의 단순 거리 대신 실제 양자화한 RGB/255의 Euclidean distance로 수행한다. 기본 최소 거리는 흰 배경으로부터 0.4, 한 view의 서로 다른 물체 사이 0.3, view 사이 동일 물체의 색상에 대해 0.15이다. A와 B는 clean의 세 물체 색상을 모두 바꾸고, B는 A와도 다르게 만든다. 최대 10,000회 시도 후 실패하면 오류를 낸다.

Clean RGB는 block/agent/goal 순서로 (119, 136, 153), (65, 105, 225), (144, 238, 144)이다. 양자화 전 HSV/RGB와 renderer에 전달한 실제 RGB를 함께 저장한다. 별도 난수 stream으로 물리 상태와 무관하게 색상을 할당하도록 설계했지만, 유한한 고정 데이터에서 우연한 상관이 생길 가능성까지 제거되는 것은 아니다.

3.4 연속 corruption 강도

종류는 후보 상태 전체에 거의 균등하게 배치한 뒤 섞는다. Reset rejection 때문에 최종 채택 데이터의 비율은 조금 달라질 수 있다. 상태마다 corruption realization 하나를 저장하고 epoch가 바뀌어도 같은 것을 재사용한다. Canonical salt/pepper의 p 는 선택된 pixel 비율의 기댓값

종류	학습 범위	정확한 의미
Gaussian	$\sigma \sim U(1, 8)$	uint8 intensity 단위의 독립적인 channel 별 영평균 noise. $[0, 255]$ clip 후 uint8 변환.
Salt/pepper	$p \sim U(0.001, 0.02)$	Pixel마다 한 번 주춤. $p/2$ 확률로 white, $p/2$ 로 black, 나머지는 유지.
Blur	$\sigma \sim U(0.3, 1.2)$	Image pixel 단위 Gaussian sigma. Kernel은 $\max(3, 2 \text{round}(3\sigma) + 1)$ 이며 sigma 0은 identity.

표 3: 대표 실제 데이터의 visual sanity check가 필요한 후보 기본값.

이다. 이미 흰색이나 검은색인 pixel은 선택되어도 외관이 그대로일 수 있다. 기존 snp1/snp5는 salt/pepper 좌표를 각각 복원추출하던 동작을 보존한다. 따라서 새 canonical p 와 같은 의미로 해석하면 안 된다. dark는 기존 평가 호환성을 위해 남아 있지만 기본 Track A 학습에는 포함하지 않는다.

생성기는 범위의 낮은/중간/높은 강도와 50% mask를 보여주는 contact sheet를 먼저 만든다. 전체 생성에는 generation.ranges_reviewed=true가 필요하다. 이 설정은 미리보기를 검토했다는 표시이며, 모든 random mask에서 모든 물체가 보인다는 자동 보장은 아니다.

3.5 Split, 상태 선택, metadata

Trajectory 단위로 약 70/15/15% train/validation/test를 한 번 배정한다. Validation/test에 각각 적어도 하나의 trajectory를 남기므로, 작은 smoke 데이터의 비율은 정확히 일치하지 않을 수 있다. 기본 frame stride는 raw frame 다섯 개이며, 상태 수 상한을 추가로 설정할 수 있다. 선택 결과와 split을 저장하고 파일 순서로 다시 계산하지 않는다. 같은 상태의 모든 view는 같은 split에 속한다.

유효 길이 L , raw timestep t 에 대해

$$u = t / \max(L - 1, 1), \quad \text{segment}(u) = \begin{cases} \text{early}, & 0 \leq u < 1/3, \\ \text{middle}, & 1/3 \leq u < 2/3, \\ \text{late}, & 2/3 \leq u \leq 1 \end{cases} \quad (9)$$

로 구간을 정의한다. Middle이 항상 더 어렵다고 가정하지 않고 실제 결과로 판단한다. Manifest에는 source 경로, 선택 ID/timestep, seed, cap, 색상/corruption 설정, schema, metadata hash와 코드 정보를 기록한다. 무결성 검사에서는 개수, 이미지 경로, 검증 상태, trajectory 집합의 분리를 확인한다.

4 Encoder와 공유 reconstruction decoder

4.1 Visible token만 처리하는 양방향 encoder

이미지를 $\tilde{x} = x/127.5 - 1$ 로 정규화한다. 224×224 RGB 입력에서 16×16 patch를 만들면 $P = 196$, patch당 pixel scalar 수는 $Q = 768$ 이다. Patch p 를 $\pi_p(\tilde{x}) \in \mathbb{R}^Q$ 라 하면 입력 token은

$$t_p = W_E \pi_p(\tilde{x}) + b_E + e_p \quad (10)$$

이다. $r = 0.5$ 에서 random permutation으로 $|V| = \lfloor P(1 - r) \rfloor = 98$ 개를 남긴다. `ids_keep`로 patch와 원래 위치의 positional embedding을 함께 선택한다. Visible patch를 새로운 위치 $0, 1, \dots$ 로 재번호화하지 않는다.

Encoder는 6개의 pre-normalized transformer block, 6개 attention head, 384차원 embedding, 4d feed-forward width, GELU, dropout 0, 최종 LayerNorm을 사용한다. Self-attention은 공간적으로 양방향이며 기존 causal predictor mask를 재사용하지 않는다. Render update에서는 clean/A/B가 같은 mask를 사용하고, corruption update에서는 clean/corrupt가 같은 mask를 사용한다. 따라서 정렬하는 latent token은 동일한 공간 위치에 대응한다.

4.2 Mask token 삽입과 위치 복원

Visible latent를 decoder의 192차원으로 projection하고 학습 가능한 mask token을 추가한다. `ids_restore`로 raster 순서를 복원한 뒤 decoder 위치 정보를 더한다.

$$H_0 = \text{Restore}([W_D Z_V + b_D; m, \dots, m]) + E_D. \quad (11)$$

Decoder는 2개 block과 6개 head를 사용하고, $\hat{X} \in \mathbb{R}^{B \times 196 \times 768}$ 을 출력한다. 모든 patch를 예측하지만 주 reconstruction loss는 가린 patch에서만 계산한다.

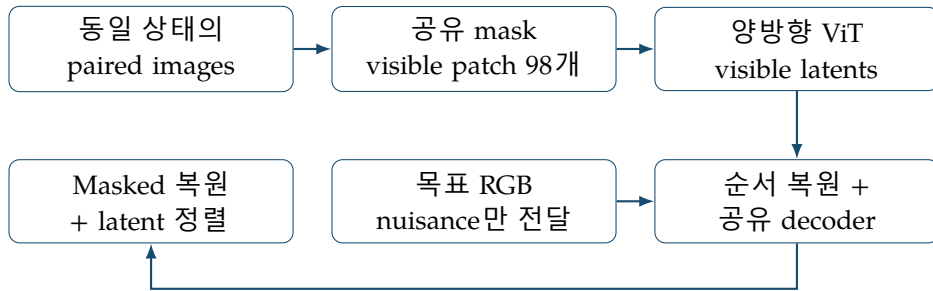


그림 1: 학습 정보 흐름. Physical-state label은 pairing 검증과 학습 후 probe에서만 사용한다.

4.3 연속 목표 RGB conditioning

물체 $o \in \{b, a, g\}$ 의 RGB에 공유 MLP를 적용하고 type embedding을 더한다.

$$n_o = f_{\text{RGB}}(c_o) + e_{\text{type}}(o), \quad n_o \in \mathbb{R}^{64}. \quad (12)$$

기본 additive 모드에서는 block/agent/goal 순서로 concat한다.

$$n_{\text{global}} = W_n[n_b; n_a; n_g] + b_n \in \mathbb{R}^{192}, \quad H_{0,p} \leftarrow H_{0,p} + n_{\text{global}}. \quad (13)$$

Optional cross-attention에서는 세 token을 decoder 차원으로 projection해 $N \in \mathbb{R}^{3 \times 192}$ 로 만든 뒤 다음을 적용한다.

$$H' = H + \text{SA}(\text{LN}(H)), \quad (14)$$

$$H'' = H' + \text{CA}(\text{LN}(H'), N, N), \quad (15)$$

$$H_{\text{next}} = H'' + \text{MLP}(\text{LN}(H'')). \quad (16)$$

Additive block은 cross-attention 항을 사용하지 않는다. 전달하는 RGB는 항상 목표 영상의 색상이다. Denoising에서도 η_0 를 전달하며, corruption metadata는 전달하지 않는다.

4.4 Tensor shape와 optional CLS

Tensor	기본 shape
입력 이미지	$[B, 3, 224, 224]$
Patch pixel	$[B, 196, 768]$
전체 encoder 출력	$[B, 196, 384]$
Masked encoder 출력	$[B, 98, 384]$
Decoder visible projection	$[B, 98, 192]$
복원한 decoder token	$[B, 196, 192]$
예측 patch pixel	$[B, 196, 768]$

표 4: 차원은 config로 변경할 수 있으며 표는 기본 설정 기준이다.

CLS를 사용하면 내부 처리에 token이 추가되지만 공개 `forward(images)`는 항상 patch token만 반환한다. 별도 feature API에서 CLS를 요청할 수 있다. Optional global 경로는 CLS를 decoder 차원으로 projection하고 위치별로 broadcast한 뒤 같은 decoder block/head를 재사용한다. 완전히 별개의 decoder network를 하나 더 학습하는 구조는 아니다.

5 정확한 학습 목적함수

5.1 Masked reconstruction의 정의

m_{ip} 를 가린 patch이면 1, 아니면 0이라 하자. Source view u 와 target view v 에 대해

$$\ell(u \rightarrow v) = \frac{\sum_{i=1}^B \sum_{p=1}^P m_{ip} \left[Q^{-1} \sum_{j=1}^Q (\hat{X}_{ipj}^{u \rightarrow v} - X_{ipj}^v)^2 \right]}{\sum_{i,p} m_{ip}}, \quad \hat{X}^{u \rightarrow v} = D_\phi(Z^u, \eta_v) \quad (17)$$

로 정의한다. 기본 target X^v 는 $[-1, 1]$ 이미지의 patch pixel이다. `loss.norm_pix_loss=true`이면 각 target patch의 평균과 모집단 분산에 10^{-6} 을 더한 값으로 추가 정규화한다. 이는 이미지 정규화와 다르다. 이때 정규화한 예측값만으로 원래 RGB를 정할 수 없으므로 현재 구현은 해당 reconstruction preview를 빈 패널로 표시한다.

5.2 Clean MAE baseline

$$\mathcal{L}_{\text{clean}} = \ell(0 \rightarrow 0). \quad (18)$$

같은 conditional decoder 구조에 고정 clean RGB를 제공한다. 일반적인 작업별 masked reconstruction만으로 충분한지를 확인하는 대조군이다.

5.3 Render 목적함수: clean 중심 양방향 복원

$q \in \{A, B\}$ 마다

$$\mathcal{L}_{\text{cross},q} = \frac{1}{2} [\ell(0 \rightarrow q) + \ell(q \rightarrow 0)], \quad (19)$$

$$\mathcal{L}_{\text{inv},q} = \frac{1}{B|V|d} \sum_{i,p \in V_{i,k}} (Z_{ipk}^0 - Z_{ipk}^q)^2 \quad (20)$$

를 계산한다. 각 render update 안에서 A와 B를 동일 비중으로 평균한다.

$$\mathcal{L}_{\text{cross}} = \frac{1}{2} (\mathcal{L}_{\text{cross},A} + \mathcal{L}_{\text{cross},B}), \quad (21)$$

$$\mathcal{L}_{\text{r-inv}} = \frac{1}{2} (\mathcal{L}_{\text{inv},A} + \mathcal{L}_{\text{inv},B}), \quad (22)$$

$$\boxed{\mathcal{L}_{\text{render}} = \mathcal{L}_{\text{cross}} + \lambda_r \mathcal{L}_{\text{r-inv}}}. \quad (23)$$

세 view를 encode하고 네 방향의 reconstruction을 계산한다. Invariance의 양쪽 encoder 출력 모두로 gradient가 흐른다. Frozen teacher나 stop-gradient는 없다. 기본 $\lambda_r = 1$ 은 출발점이며 최적 가중치가 입증된 것은 아니다.

5.4 Corruption 목적함수: 손상 입력에서 clean 복원

$$\mathcal{L}_{\text{denoise}} = \ell(c \rightarrow 0), \quad (24)$$

$$\mathcal{L}_{\text{c-inv}} = \frac{1}{B|V|d} \sum_{i,p \in V_{i,k}} (Z_{ipk}^0 - Z_{ipk}^c)^2, \quad (25)$$

$$\boxed{\mathcal{L}_{\text{corrupt}} = \mathcal{L}_{\text{denoise}} + \lambda_c \mathcal{L}_{\text{c-inv}}}. \quad (26)$$

기본 $\lambda_c = 1$ 이다. Decoder에는 η_0 를 주고 noise 종류, 강도, seed는 주지 않는다. 가린 clean-target patch만 복원하므로, visible pixel까지 모두 복원하는 full-image denoising loss는 아니다. Visible 영역의 representation은 별도 alignment 향이 정렬한다.

5.5 Optional global reconstruction과 collapse 진단

활성화한 복원 방향마다 CLS 경로는 전체 patch의 MSE를 추가한다.

$$\ell_g(u \rightarrow v) = \frac{1}{BPQ} \sum_{i,p,j} (D_{g,\phi}(z_{\text{CLS}}^u, \eta_v)_{ipj} - X_{ipj}^v)^2. \quad (27)$$

기존 목적함수와 같은 방향 집합에서 평균한 $\mathcal{L}_g$ 에 λ_g 를 곱해 더한다. 기본 실험에서는 CLS, global reconstruction을 끄고 $\lambda_g = 0$ 으로 둔다.

Latent agreement만 최소화하면 상수 또는 정보가 적은 표현도 해가 될 수 있다. Reconstruction이 이를 견제하지만 collapse를 막는 수학적 보장은 아니다. 원래/가중 loss, latent norm, 거리, 상태 사이 분산, physical probe를 함께 확인해야 한다. State probe가 나쁜데 latent 거리만 작아진 경우를 성공으로 보지 않는다.

모드	목적함수	Encoder view/update	Decoder pass/update
clean_mae	$\mathcal{L}_{\text{clean}}$	1	1
render_only	$\mathcal{L}_{\text{render}}$	3	4
corruption_only	$\mathcal{L}_{\text{corrupt}}$	2	1
robust_alternating	Render/corruption 교대	3, 다음 2	4, 다음 1

표 5: CLS를 끈 기본 상태에서 표본당 계산 횟수. Validation과 preview는 제외한다.

6 최적화 알고리즘과 예산

6.1 네 가지 학습 모드

각 실험 run은 자체 encoder/decoder를 초기화한다. 공유 decoder란 한 run 안에서 목적함수들이 같은 decoder를 쓴다는 뜻이다. 서로 다른 실험 run 사이의 weight를 공유한다는 뜻은 아니다.

6.2 Alternating 알고리즘

짝수 총예산 T 와 전체 parameter에 대한 하나의 AdamW optimizer를 만든다. $t = 0, \dots, T - 1$ 에 대해 다음을 반복한다.

1. 채택한 training state에서 B 개 index를 균등 복원추출한다.
2. 해당 offline view와 목표 RGB를 읽는다. Training batch에는 state label이 없다.
3. 이번 update에서 사용할 새로운 shared spatial mask를 생성한다.
4. t 가 짝수이면 $\mathcal{L}_t = \mathcal{L}_{\text{render}}$, 홀수이면 $\mathcal{L}_t = \mathcal{L}_{\text{corrupt}}$ 로 둔다.
5. Gradient를 비우고 $\nabla_{\theta, \phi} \mathcal{L}_t$ 를 계산한다. 비정상 loss/gradient는 중단하고 global gradient norm을 1로 clip한 뒤 AdamW를 한 번 실행한다.
6. $\text{global_step} = t + 1$ 로 기록하고 주기적으로 validation과 저장을 수행한다.

한 cycle은 optimizer update 두 번이다. Clean MAE update를 세 번째로 추가하지 않는다. 각 update에서 clean encoding을 다시 계산하므로 corruption update가 직전 render update의 오래된 feature를 재사용하지 않는다. 공유 moment state를 사용하는 순차 AdamW update는 두 loss를 더해 한 번 update하는 것과 동일하지 않다.

Training의 epoch는 누적 추출 상태 수를 training 데이터 크기로 나눈 값이다. Shuffle한 전체 dataset pass를 뜻하지 않는다. 현재 validation은 validation record 앞부분의 제한된 batch와 고정 mask RNG seed 9871을 사용한다. 따라서 이 loss는 진단용이며 held-out 상태 전체의 평가가 아니다. 주 비교에서는 고정 update 예산의 마지막 checkpoint를 사용해 test 기반 checkpoint 선택을 피한다.

6.3 공정한 비교의 의미

모드 간 architecture, mask, optimizer, dataset, 전체 update 수를 맞춘다. $T = 10,000$ 이면 alternating은 render 5,000회와 corruption 5,000회이다. 이는 *update-matched* 비교이지 compute-

설정	기본값
전체 optimizer update	10,000
Batch 크기	상태 16개
AdamW learning rate / weight decay	10^{-4} / 0.05
Gradient norm 상한	1.0
Precision / LR schedule	float32 / constant
Validation / checkpoint 주기	100 update / 100 update
Validation 상한	4 batch
이미지 기록 주기	100 update
Scalar 기록 주기	목적함수별 10 update

표 6: 튜닝된 최적값이 아닌 기본값. Alternating에서는 두 목적함수를 모두 기록한다.

matched 비교가 아니다. CLS가 없을 때 상태 추출 한 번당 평균 encoder/decoder view 수는 alternating 2.5/2.5, clean MAE 1/1이다. Image-pass count와 실행 시간도 보고한다. Compute-matched 비교는 후속 별도 protocol이며 resume 이후 elapsed time은 현재 process 구간의 시간이다.

7 Frozen representation 평가

7.1 Physical-state ridge probe

Encoder를 evaluation mode로 고정한다. Mask 없이 patch token을 평균한다.

$$f_i = P^{-1} \sum_{p=1}^P E_{\theta}(x_i)_p \in \mathbb{R}^{384}. \quad (28)$$

Canonical rendered state에서 다음 target을 만든다.

$$y_i = (a_x, a_y, b_x, b_y, \cos \theta, \sin \theta, v_{a,x}, v_{a,y}) \in \mathbb{R}^8. \quad (29)$$

Feature/target 평균과 표준편차는 clean train에서만 구하고, 표준편차 하한은 10^{-6} 으로 둔다. 표준화한 행렬을 F, Y 라 하면

$$W_{\alpha} = \arg \min_W \|FW - Y\|_F^2 + \alpha \|W\|_F^2 = (F^{\top} F + \alpha I)^{-1} F^{\top} Y. \quad (30)$$

표본 수가 feature 차원보다 작으면 동등한 dual form을 푼다. Centering이 intercept를 처리한다. $\alpha \in \{0.001, 0.01, 0.1, 1, 10\}$ 중 clean validation의 standardized target MSE가 가장 작은 값을 선택한다. Shifted test 전에 readout과 모든 정규화 통계를 고정한다.

7.2 Metric과 해석

조건 c 의 test sample 수가 n 일 때 주 aggregate metric은

$$\text{SMSE}(c) = \frac{1}{8n} \sum_{i,j} \left(\frac{\hat{y}_{ij}^c - y_{ij}}{\sigma_{y,j}^{\text{train}}} \right)^2 \quad (31)$$

이다. 추가로 원래 단위의 agent-position MSE, block-position MSE, velocity MSE, target별 R^2 , wrapped angle MAE를 보고한다.

$$\hat{\theta}_i = \text{atan2}(\hat{y}_{i,\sin}, \hat{y}_{i,\cos}), \quad (32)$$

$$\text{MAE}_\theta = \frac{1}{n} \sum_i \left| \text{atan2}(\sin(\hat{\theta}_i - \theta_i), \cos(\hat{\theta}_i - \theta_i)) \right|. \quad (33)$$

각도는 radian과 degree로 모두 기록한다. Test target 분산이 사실상 0이면 R^2 를 정의할 수 없어 null로 저장한다. 음수 R^2 도 가능하다. 매우 작은 target 분산과 0에 가까운 예측 sine/cosine은 해석에 주의한다.

구현된 clean-to-shift degradation은

$$\Delta(c) = \text{SMSE}(c) - \text{SMSE}(0) \quad (34)$$

이다. 평가 latent distance는 전체 이미지의 mean-pooled feature 사이 RMS이다. Visible patch token을 정렬하는 학습 loss와 같은 양이 아니다. Raw latent distance나 RGB probe 결과만으로 유용한 invariance를 입증할 수 없다.

7.3 Test 조건

조건	Intervention
default, colorA, colorB	저장된 held-out view
redBlock, redAgent, redAnchor	해당 물체 RGB만 (255, 0, 0)으로 변경. Anchor는 goal marker
random_inside	학습 sampling 범위 안에서 새 색상을 생성
random_outside	기본 saturation [0.91, 1], value [0.86, 0.95]. 같은 거리 검사 적용
Gaussian grid	uint8 단위 $\sigma \in \{0, 4, 8, 12\}$
Salt/pepper grid	$p \in \{0, 0.01, 0.02, 0.04\}$
Blur grid	Pixel 단위 $\sigma \in \{0, 0.6, 1.2, 2\}$

표 7: 구현된 평가 protocol. 기본 evaluation seed는 100이며 0 강도는 identity control이다.

기본 grid에는 학습 범위 내부와 더 강한 외삽 조건이 함께 있다. 학습 색상 범위를 변경하면 outside 범위와 겹치지 않는지 다시 확인한다. 추가 평가 view는 별도로 생성하므로 training sample당 정확히 네 view라는 규칙은 유지된다. Full/early/middle/late에서도 같은 clean-trained probe를 사용한다. 재렌더링 state는 저장된 canonical과 비교하며, 주 평가에서 조건마다 probe를 다시 학습하지 않는다.

7.4 Nuisance 진단과 reference

RGB ridge probe는 색상이 변하는 A/B view로 학습하며 block/agent/goal을 각각 평가한다. Clean 색상만 사용하면 target이 상수여서 유의미한 probe가 되지 않는다. RGB 예측력이 낮아지는 것은 scene 정보가 유지될 때만 좋은 신호이다. 선형 probe가 약하다는 이유만으로 통계적 독립을 증명할 수는 없다.

Corruption-type probe는 one-hot label의 ridge regression 출력에 argmax를 적용한다. Logistic classifier는 아니다. Accuracy, 균등 chance, train majority baseline을 보고한다. Strength regression은 단위가 다른 종류를 섞지 않도록 종류마다 따로 학습한다. 표본이 부족하면 그 사실을 명시한다.

Reference는 random scratch, clean MAE, render-only, corruption-only, alternating, frozen pre-trained DINOv2 [2]이다. DINO는 평가만 한다. 기존 wrapper는 DINO에 196px/patch14 입력을 넣어 196 token을 얻고, scratch는 224px/patch16을 사용한다. 전처리를 유지하고 차이를 metadata에 남긴다. DINO의 외부 사전학습 데이터와 학습 이력도 다르므로 엄격한 equal-data 비교는 아니다.

8 실험 계획법과 판단 기준

8.1 검증할 가설

- H1: 색상 invariance.** Render-only가 clean MAE보다 색상 shift의 degradation을 줄이면서 clean 기하 정확도를 실용적으로 수용할 수 있는 범위 안에 유지한다.
- H2: Corruption robustness.** Corruption-only가 clean MAE보다 corruption curve를 개선하며, 특히 학습 강도 범위에서 효과를 보인다.
- H3: 두 효과의 결합.** Alternating이 고정 update 예산에서 두 종류의 이득을 유지하며, 단순한 latent norm 축소만 일으키는 것은 아니다.
- H4: Decoder ablation.** Cross-attention이나 CLS global 경로가 기하 정보와 invariance의 trade-off를 바꿀 수 있다. 개선을 전제하지 않는다.

8.2 주 실험 matrix

수행할 실험. 검토한 하나의 manifest, split seed, sampling 범위와 evaluation seed를 고정한다. 네 objective에 training seed {0, 1, 2}를 적용해 총 12회 학습한다. Additive conditioning, CLS off, mask 0.5, $\lambda_r = \lambda_c = 1$, 10,000 update를 사용하고 objective 간 같은 seed끼리 비교한다. Random encoder와 frozen DINO도 같은 trajectory/probe protocol로 평가한다. 현재 random reference의 초기화는 evaluation.seed를 사용한다. 이를 바꾸면 새 평가 view도 달라진다. 따라서 기본 reference는 evaluation seed를 고정하고, 다른 protocol을 training replicate처럼 한데 집계하지 않는다.

통제 요소	계획
데이터	같은 채택 상태, manifest, 네 저장 view, split ID
초기화	학습 objective 네 개에 동일 training seed 집합
최적화	같은 구조, batch, LR, optimizer, clipping, update 수
Checkpoint	고정 예산 마지막 결과. Test 기반 선택 금지
Probe	같은 mean pooling, clean-only fitting, validation 선택 grid
Test 난수	비교하는 학습 run에서 evaluation seed와 grid 고정
보고	조건별/segment별 오차와 training seed 사이 변동

표 8: 앞으로 수행할 주 비교 설계. 이 matrix는 아직 실행하지 않았다.

8.3 실행 순서와 ablation

1. 실제 source state와 rejection 비율을 점검하고 contact sheet를 검토한 후 manifest를 확정한다.

2. CIRCE에서 온라인 W&B를 사용하는 2-update smoke run으로 live metric, 종료 상태, 로컬 JSONL, checkpoint와 reconstruction 이미지를 확인한다.
3. Clean MAE와 alternating의 seed 한 개로 engineering pilot을 수행한다. 오류 탐지용이며 이 결과만으로 강한 연구 주장을 하지 않는다.
4. 사전에 정한 네 objective, 세 seed의 주 matrix를 실행한다.
5. 고정한 protocol로 모든 선택 checkpoint와 두 reference를 평가한다.
6. 주 matrix 이후 controlled ablation을 수행한다. Invariance 0 대 nonzero, additive 대 cross-attention, CLS/global off 대 on, mask ratio를 비교하며 가능하면 한 비교에서 한 요인만 바꾼다.

탐색용 후보는 $\lambda \in \{0, 0.1, 1\}$, $r \in \{0.25, 0.5, 0.75\}$ 이다. 이는 아직 실행하거나 튜닝한 값이 아니다. Render/corruption 가중치는 하나를 고정한 채 다른 하나를 바꾸는 비교부터 시작한다. 좋은 결과만 남기지 말고 모든 시도한 설정을 기록한다.

8.4 어떤 결과가 가설을 지지하는가?

수행할 실험. Shift error와 함께 clean error를 항상 보고한다. Clean 성능이 나빠져서 degradation만 작아지는 경우는 반드시 유용한 개선이 아니다. 최종 test 전에 training/validation 근거와 작업 단위를 사용해 허용 가능한 clean 기하 오차 증가량을 정하고 기록한다. 현재 구현에 그러한 수치 threshold가 확정되어 있는 것은 아니다.

Clean 위치/각도 정확도를 유지하고, 색상과 corruption 양쪽에서 오차를 줄이며, 여러 training seed에서 일관된 결과를 얻으면 가설을 지지한다. 학습한 corruption 강도에서만 좋아지면 범위 안 interpolation robustness의 근거이며 무제한 일반화의 근거는 아니다. RGB 예측력과 geometry probe가 함께 나빠지면 정보 손실을 의심한다. Reconstruction은 좋은데 probe가 나쁘면 pixel 복원만으로 유용한 representation이 확보되지 않은 것이다.

8.5 통계 보고

현재 집계기는 서로 다른 training seed의 평균과 표본 표준편차를 표시한다. Confidence interval을 계산하는 것은 아니다. K 개 run에 대해

$$\bar{m} = K^{-1} \sum_k m_k, \quad s_m = \sqrt{\frac{\sum_k (m_k - \bar{m})^2}{K - 1}} \quad (35)$$

를 사용한다. 세 seed는 첫 안정성 확인에는 유용하지만 강한 통계적 검정력을 보장하지 않는다. 같은 trajectory의 frame은 상관되어 있다. 현재 run metric은 frame-weighted이므로 채택한 frame이 많은 trajectory가 더 기여한다. 후속 trajectory-balanced 또는 bootstrap 분석에서는 frame을 독립적으로 뽑지 말고 trajectory 전체를 재표집해야 한다. 이를 위해서는 현재 aggregate 보고 외에 per-sample prediction을 저장/내보내는 기능이 추가로 필요하다. 기존 SD bar를 신뢰구간이라고 표시하지 않는다.

9 W&B 기록, 재현성, run 관리

구현된 내용. 기본 W&B project는 `adajepa_trackA`이며 run 이름에는 `objective, conditioning, mask, seed`가 들어간다. 학습 그래프의 X축은 `train/global_step`이다. W&B 호출 전에 로컬 JSONL을 flush하므로 service 문제가 생겨도 scalar 기록이 남는다. Config, manifest hash, source hash, git SHA와 dirty 여부를 저장한다. 조건별 probe table, 색상 bar chart, 종류별 strength curve도 기록하며 모두 로컬 결과가 있다.

`render/cross_mae`, `render/invariance`, `weighted invariance`, `corruption denoising/invariance`, `clean/mae`, `total/global loss`, `latent norm/variance/distance`, `gradient norm`, `LR`, `sample count`, `image-pass count`, `elapsed time`을 확인한다. Validation은 별도 key이다. Training epoch, optimizer step, MPC step은 서로 다르다. 이 브랜치는 MPC success-rate curve를 생성하지 않는다.

Checkpoint에는 `encoder/decoder/optimizer state`, `update/cycle/sample counter`, 다음 objective, RNG state, resolved config, manifest/schema, git/source 정보, output 경로와 W&B run ID가 들어간다. Float32 constant-LR 학습이므로 AMP scaler나 scheduler는 없다. Online resume은 기존 project/entity와 run ID를 사용하며, config/manifest 불일치 또는 오래된 checkpoint는 거부한다. Offline resume은 새 offline segment를 만들되 로컬 optimizer step은 이어간다.

정상 종료와 예외 종료에서 W&B status를 구분하고 Python traceback을 로컬에 남긴다. SIGKILL이나 전원 손실에서는 종료 처리를 실행할 수 없다. 복구 CLI는 로컬 `metric/table/chart`를 명확히 이름 붙인 새 run으로 재기록한다. 기본 `cache/staging`을 output 경로에 두어 home cache 권한에 덜 의존한다. Offline 직렬화는 검증했지만 인증된 cloud 전송은 CIRCE에서 확인해야 한다.

10 구현 검증과 한계

로컬 테스트 24개가 Python 3.11.14, PyTorch 2.3.0, torchvision 0.18.0, NumPy 1.26.4, Hydra 1.3.2, W&B 0.22.3, pymunk 6.6.0에서 통과했다. 실제 PushT rerender, state/mask 무결성, 모든 objective/conditioning, 역전파, CLS 호환성, CPU resume 일치, probe와 offline W&B chart 복구를 검증했다. Python 3.9 문법은 검사했지만 실제 3.9 runtime, CUDA, CIRCE 실행데이터 학습, DINO weight 로딩은 로컬 검증 대상에 포함되지 않았다.

1. State schema가 불완전하며 정적 영상만으로 모든 state 성분을 식별할 수 없다.
2. Reconstruction과 MSE alignment는 인과적 disentanglement나 collapse 방지를 보증하지 않는다.
3. 유한한 offline 색상/noise realization이 학습 내내 재사용된다.
4. Reset rejection과 채택 trajectory 길이가 평가 모집단을 바꿀 수 있다.
5. Linear probe는 읽을 수 있는 정보의 측정이며 policy 성공률이 아니다.
6. 색상과 noise를 동시에 바꾼 일반화는 후속 실험이며 현재 입증된 결과가 아니다.
7. 기본 loss 가중치, corruption 범위, 학습 예산은 실증 검증이 필요하다.
8. AdaJEPA token shape 호환성은 기존 checkpoint와 latent 의미가 호환된다는 뜻이 아니다.

이전 planning run의 성공률을 새 encoder 실험의 결과로 제시하면 안 된다. 다음 단계의 타당한 산출물은 실험 변동과 제한을 함께 기록한 재현 가능한 representation 비교 결과이다.

A 실행 명령 순서

다음은 CIRCE의 저장소 폴더에서 실행하는 Bash 명령이다. Source/checkpoint는 실제 절대 경로로 바꾼다. 모든 option은 구현된 Hydra config에 존재한다.

```
SOURCE=/absolute/path/to/pusht_dataset
PAIRS="$PWD/data/tracka_pairs"

python generate_pusht_pairs.py --config-name tracka \
  dataset.source_path="$SOURCE" dataset.output_path="$PAIRS" \
  generation.preview_only=true

# After inspecting the exported contact sheets:
python generate_pusht_pairs.py --config-name tracka \
  dataset.source_path="$SOURCE" dataset.output_path="$PAIRS" \
  generation.preview_only=false generation.ranges_reviewed=true

# Engineering smoke run:
python train_encoder.py --config-name tracka \
  dataset.output_path="$PAIRS" training.objective=robust_alternating \
  training.total_optimizer_updates=2 training.batch_size=2 \
  logging.log_every=1

# Planned primary matrix, after obtaining compute resources:
for SEED in 0 1 2; do
  for MODE in clean_mae render_only corruption_only robust_alternating; do
    python train_encoder.py --config-name tracka \
      dataset.output_path="$PAIRS" training.objective="$MODE" \
      training.seed="$SEED" logging.group="trackA-seed${SEED}"
  done
done
```

출력된 run 경로마다 마지막 checkpoint를 평가한다.

```
python eval_encoder_probes.py --config-name tracka \
  dataset.output_path="$PAIRS" evaluation.encoder=checkpoint \
  evaluation.checkpoint=/absolute/path/to/run/checkpoint_latest.pt

python eval_encoder_probes.py --config-name tracka \
  dataset.output_path="$PAIRS" evaluation.encoder=random

python eval_encoder_probes.py --config-name tracka \
  dataset.output_path="$PAIRS" evaluation.encoder=dino

python aggregate_tracka_results.py \
  /absolute/path/to/probe_clean /absolute/path/to/probe_robust \
  --output-dir "$PWD/tracka_outputs/comparison"

python recover_tracka_wandb.py /absolute/path/to/run
```

Cloud 접근이 안 되면 `logging.mode=offline`을 사용한다. Resume에는 기존 비기본 `model/loss/training` 설정, `training.resume=/absolute/path/to/checkpoint_latest.pt`, 더 큰 전체 update 예산이 필요하다. Source에는 `states.pth`, 필요한 경우 `velocities.pth`, `seq_lengths.pkl`, 선택적으로 `shapes.pkl`이 있어야 한다. 실제 대규모 실행은 cluster에서 할당 받은 자원 안에서 수행한다. 이 문서는 확인하지 않은 scheduler resource option을 임의로 추가하지 않는다.

B 코드와 방법의 대응

파일	담당 내용
<code>conf/tracka.yaml</code>	기본값과 실험 option
<code>tracka/data.py</code>	Rerender, rejection, four-view record, manifest, loader
<code>tracka/model.py</code>	Encoder, 공유 decoder, mask 복원, 손실함수
<code>tracka/train.py</code>	Sampling, objective 교대, 최적화, resume
<code>tracka/probes.py</code>	Clean-trained ridge probe와 shifted 평가
<code>tracka/logging.py</code>	로컬 기록과 W&B lifecycle/chart
<code>planning/image_corruption.py</code>	Canonical 및 기존 호환 corruption
<code>aggregate_tracka_results.py</code>	Protocol 검증, CSV, seed SD 비교 그림
<code>tests/test_tracka.py</code>	Smoke 및 regression 검증
<code>docs/tracka_validation.md</code>	실제 테스트 환경 및 검증 범위

참고문헌

- [1] K. He, X. Chen, S. Xie, Y. Li, P. Dollár, and R. Girshick. *Masked Autoencoders Are Scalable Vision Learners*. arXiv:2111.06377, 2021. <https://arxiv.org/abs/2111.06377>.
- [2] M. Oquab et al. *DINOv2: Learning Robust Visual Features without Supervision*. arXiv:2304.07193, 2023. <https://arxiv.org/abs/2304.07193>.